

Lista de Exercícios: POO em JavaScript

Atributos Privados, Métodos, Classes, Herança, Polimorfismo, Getters e Setters (ES6+)

Disciplina: Programação Orientada a Objetos

Linguagem: JavaScript (ES6+)

Níveis: Médio e Alto

NÍVEL MÉDIO

1. Sistema de Gestão de Contas Bancárias

Conceitos: Atributos privados (#), Getters/Setters, Encapsulamento e Herança

1. Crie a classe **ContaBancaria**:

- Declare os atributos privados `#numeroConta`, `#titular` e `#saldo`.
- Crie um construtor que receba `numeroConta`, `titular` e um `saldoInicial` (opcional, valor padrão 0).
- Crie getters para `numeroConta`, `titular` e `saldo`. Não crie setter para o `saldo`.
- Implemente os métodos `depositar(valor)` e `sacar(valor)`. O método `sacar` deve verificar se há saldo suficiente e retornar `true` em caso de sucesso e `false` caso contrário.

2. Crie a classe **ContaEspecial** (herda de **ContaBancaria**):

- Adicione o atributo privado `#limite`.
- Crie getter e setter para `#limite` (com validação para não aceitar limite negativo).
- Sobrescreva o método `sacar(valor)` para permitir saques onde o saldo final pode ficar negativo até o valor do `#limite`.

2. Controle de Frota de Veículos

Conceitos: Herança, Sobrescrita de Métodos e Polimorfismo

1. Crie a classe **Veiculo**:

- Atributos privados: `#marca`, `#modelo`, `#valorBaseDiaria`.
- Crie os getters necessários.
- Crie o método `calcularAluguel(dias)`, que retorna `dias * #valorBaseDiaria`.

2. Crie as subclasses que estendem **Veiculo**:

- **CarroPasseio**: possui o atributo privado `#possuiArCondicionado` (booleano). Sobrescreva `calcularAluguel(dias)` adicionando uma taxa extra de 10% no total caso tenha ar-condicionado.
- **Caminhao**: possui o atributo privado `#capacidadeToneladas`. Sobrescreva `calcularAluguel(dias)` adicionando R\$ 50,00 por dia para cada tonelada de capacidade.

3. Processamento Polimórfico:

- Crie uma função `imprimirResumoAluguel(veiculos, dias)` que recebe um Array de veículos e imprime no console o modelo e o valor do aluguel de cada um utilizando **polimorfismo**.

3. Motor de Folha de Pagamento

Conceitos: Getters/Setters com validação, Polimorfismo e Manipulação de Arrays

1. Crie a classe **Funcionario**:

- Atributos privados: `#nome`, `#cpf`, `#salarioBase`.
- Valide o `#salarioBase` no setter para que não seja inferior ao salário mínimo (ex: R\$ 1.412,00). Caso seja menor, lance um erro (`throw new Error(...)`).
- Implemente o método `calcularSalarioLiquido()` que aplica um desconto fixo de 10% de imposto sobre o `#salarioBase`.

2. Crie as subclasses:

- **Desenvolvedor**: possui o atributo privado `#nivel` ('Junior', 'Pleno' ou 'Senior'). Sobrescreva `calcularSalarioLiquido()` aplicando um bônus antes do imposto (Junior: +5%, Pleno: +15%, Senior: +30%).
- **Gerente**: possui o atributo privado `#bonusAnual`. Sobrescreva `calcularSalarioLiquido()` somando a parcela mensal do bônus (`bonusAnual / 12`) ao salário base antes de aplicar os 10% de imposto.

3. Crie a classe **FolhaPagamento**:

- Método `processar(funcionarios)`: recebe um array de objetos `Funcionario`, calcula o salário líquido de cada um via polimorfismo, exibe um contracheque formatado no console e retorna o valor total investido na folha salarial pela empresa.

4. Simulação de Batalha RPG Turno a Turno

Conceitos: Regras de negócio complexas, controle de estado, Herança e Polimorfismo

1. Crie a classe base **Personagem**:

- Atributos privados: `#nome`, `#vida`, `#danoAtaque`, `#defesa`. Getters para todos os atributos.
- Setter para `#vida` que não permita valores menores que 0. Se a vida chegar a 0, exiba uma mensagem indicando que o personagem foi derrotado.
- Crie o método `estaVivo()` que retorna `true` se `#vida > 0`.
- Crie o método `atacar(alvo)`: calcula o dano como `danoAtaque - alvo.defesa`. Se o resultado for `<= 0`, o dano mínimo aplicado deve ser 1. Reduza a vida do `alvo` com o dano calculado.

2. Crie as subclasses de **Personagem**:

- **Guerreiro**: possui o atributo `#furia` (inicia em 0). A cada ataque realizado, ganha +5 de fúria e soma esse valor ao dano causado.
- **Mago**: possui o atributo privado `#mana` (inicia em 50). Sobrescreva o método `atacar(alvo)`: se tiver pelo menos 15 de mana, consome 15 de mana e realiza um *Ataque Mágico* que **ignora a defesa do alvo**. Caso contrário, faz um ataque físico normal.
- **Arqueiro**: Sobrescreva `atacar(alvo)` com uma mecânica de *Acerto Crítico*: use `Math.random()` para dar 30% de chance de dobrar o `danoAtaque` naquela rodada.

3. Crie a classe **Arena**:

- Implemente o método `iniciarCombate(p1, p2)`: executa um loop (`while`) onde `p1` e `p2` se atacam alternadamente até que um dos dois não esteja mais vivo. A cada turno, exiba no console a ação executada e a vida restante de ambos.